

# Design Pattern + Algorithm Design

## 1. Objective

ระบบ Promotion Engine ต้องถูกออกแบบให้รองรับโจทย์หลัก 3 เรื่อง:

1. รองรับ Promotion หลายรูปแบบ
2. รองรับ Promotion แบบซ้อนกัน
3. เพิ่ม Promotion ใหม่ได้โดยไม่กระทบ Logic เดิม

ปัญหาที่ต้องหลีกเลี่ยงคือการเขียน Logic แบบ Hardcode หรือ If-Else ขนาดใหญ่ เช่น:

```
if product_id == 1:
    discount = price * 10%

if product_id == 2:
    discount = 100
```

เพราะเมื่อมี Promotion ใหม่ เช่น Buy X Get Y หรือ Free Shipping ระบบจะต้องแก้ Code เดิมตลอด ทำให้เสี่ยงกระทบ Logic เก่า

ดังนั้นควรออกแบบเป็น:

```
Rule-based Promotion Engine
+ Strategy Pattern
+ Factory Pattern
+ Specification Pattern
+ Chain / Pipeline Execution
+ Repository Pattern
```

## 2. Design Pattern Overview

| Pattern                    | ใช้ทำอะไร                                     | เหตุผล                                      |
|----------------------------|-----------------------------------------------|---------------------------------------------|
| Strategy Pattern           | แยก Logic การคำนวณ Promotion แต่ละประเภท      | เพิ่ม Promotion ใหม่โดยไม่แก้ Flow หลัก     |
| Factory / Registry Pattern | เลือก Strategy จาก <code>action_type</code>   | ลด If-Else ใน Engine                        |
| Specification Pattern      | ตรวจสอบ Condition ว่า Promotion ใช้ได้หรือไม่ | แยก Eligibility ออกจาก Discount Calculation |
| Chain / Pipeline Pattern   | Apply Promotion ตามลำดับ                      | รองรับ Priority และ Stacking                |
| Policy Pattern             | ควบคุม Stacking, Exclusive, Conflict          | แยก Rule การซ้อนออกจาก Strategy             |
| Repository Pattern         | แยก Database Access                           | Test ง่ายและไม่ผูก Engine กับ DB            |
| Context Object Pattern     | เก็บ State ระหว่างคำนวณ                       | Debug และ Audit ได้ง่าย                     |

### 3. Recommended Pattern Combination

Pattern ที่เหมาะที่สุดไม่ใช่ Pattern เดียว แต่เป็นการใช้ร่วมกัน:

```
Promotion Engine
  → Condition Evaluator
  → Promotion Sorter
  → Stacking Policy
  → Strategy Factory
  → Promotion Strategy
  → Calculation Context
```

แนวคิดคือ:

```
Engine คุม Flow
Strategy คำนวณ Discount
Specification ตรวจสอบ Condition
Policy คุม Stacking
Repository โหลดข้อมูล
Context เก็บผลลัพธ์
```

## 4. Architecture of Pattern Usage

flowchart TD

A[Pricing Service] --> B[Promotion Engine]

B --> C[Condition Evaluator]

B --> D[Promotion Sorter]

B --> E[Stacking Policy]

B --> F[Strategy Factory]

F --> G[Percentage Discount Strategy]

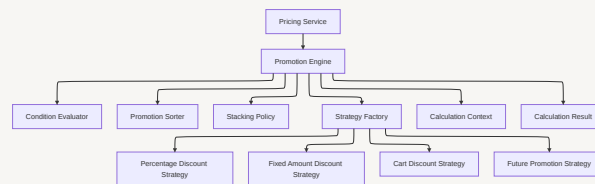
F --> H[Fixed Amount Discount Strategy]

F --> I[Cart Discount Strategy]

F --> J[Future Promotion Strategy]

B --> K[Calculation Context]

B --> L[Calculation Result]



## 5. Strategy Pattern

### 5.1 ใช้ทำอะไร

ใช้แยก Logic การคำนวณ Promotion แต่ละประเภทออกจากกัน

ตัวอย่าง Promotion Type:

```
PERCENTAGE_DISCOUNT
FIXED_AMOUNT_DISCOUNT
CART_PERCENTAGE_DISCOUNT
CART_FIXED_AMOUNT_DISCOUNT
```

```
BUY_X_GET_Y  
FREE_SHIPPING
```

แต่ละ Type มี Strategy ของตัวเอง:

```
PercentageDiscountStrategy  
FixedAmountDiscountStrategy  
CartPercentageDiscountStrategy  
CartFixedDiscountStrategy  
BuyXGetYStrategy  
FreeShippingStrategy
```

## 5.2 ทำไมต้องใช้ Strategy Pattern

เพราะ Promotion แต่ละแบบมีสูตรคำนวณไม่เหมือนกัน

ตัวอย่าง:

| Promotion       | Logic                               |
|-----------------|-------------------------------------|
| ลด 10%          | <code>amount * percent / 100</code> |
| ลด 100 บาท      | <code>fixed_amount</code>           |
| Buy 2 Get 1     | ตรวจจำนวนสินค้าและเลือกตัวที่แถม    |
| Free Shipping   | ลดจากค่าส่ง                         |
| Bundle Discount | ตรวจสินค้าหลายตัวรวมกัน             |

ถ้าเอาทุกอย่างไปเขียนใน Function เดียว จะเกิดปัญหา:

- If-Else เยอะ
- เพิ่ม Promotion ใหม่แล้วเสี่ยงพังของเดิม
- Test ยาก
- Debug ยาก
- ไม่สอดคล้องกับ Open/Closed Principle

## 5.3 Strategy Interface Concept

```
PromotionStrategy
- Apply(context, promotion) -> PromotionResult
```

Promotion Engine ไม่จำเป็นต้องรู้รายละเอียดการคำนวณของแต่ละ Promotion Engine รู้เพียงว่า:

```
Strategy ทุกตัวต้อง Apply ได้
```

## 5.4 Example Strategy Responsibility

### PercentageDiscountStrategy

รับผิดชอบ:

```
คำนวณส่วนลดแบบเปอร์เซ็นต์
ตรวจ max discount
cap discount ไม่ให้เกินยอดคงเหลือ
```

Formula:

```
discount = current_amount * value_basis_points / 10000
actual_discount = min(discount, remaining_amount)
```

ตัวอย่าง:

```
amount = 1000 บาท
discount = 10%
discount = 100 บาท
final = 900 บาท
```

### FixedAmountDiscountStrategy

รับผิดชอบ:

คำนวณส่วนลดแบบจำนวนเงินคงที่  
cap discount ไม่ให้เกินยอดคงเหลือ

Formula:

```
discount = fixed_amount  
actual_discount = min(discount, remaining_amount)
```

ตัวอย่าง:

```
amount = 500 บาท  
discount = 100 บาท  
final = 400 บาท
```

## 6. Factory / Registry Pattern

### 6.1 ใช้ทำอะไร

ใช้เลือก Strategy ตาม `action_type`

ตัวอย่าง:

```
PERCENTAGE_DISCOUNT → PercentageDiscountStrategy  
FIXED_AMOUNT_DISCOUNT → FixedAmountDiscountStrategy  
BUY_X_GET_Y → BuyXGetYStrategy
```

### 6.2 ทำไมต้องใช้ Factory

เพื่อไม่ให้ Promotion Engine ต้องเขียน If-Else เพื่อเลือก Logic เอง

ไม่ดี:

```
if action_type == PERCENTAGE_DISCOUNT:  
    calculate percentage
```

```
else if action_type == FIXED_AMOUNT_DISCOUNT:
    calculate fixed amount

else if action_type == BUY_X_GET_Y:
    calculate bogo
```

ดี:

```
strategy = strategy_factory.get(action_type)
strategy.apply(context, promotion)
```

## 6.3 เพิ่ม Promotion ใหม่ทำอย่างไร

เช่น เพิ่ม `BUY_X_GET_Y`

ต้องทำแค่:

1. สร้าง BuyXGetYStrategy
2. เพิ่ม action\_type = BUY\_X\_GET\_Y
3. Register Strategy เข้า Factory
4. เพิ่ม Validation
5. เพิ่ม Unit Test

ไม่ต้องแก้ Calculation Flow หลักของ Promotion Engine

## 7. Specification Pattern

### 7.1 ใช้ทำอะไร

ใช้ตรวจว่า Promotion ใช้ได้กับ Order หรือ Item นี้หรือไม่

ตัวอย่าง Condition:

```
PRODUCT_ID = 1
CATEGORY_ID IN [10, 11]
MIN_ORDER_AMOUNT >= 1000
COUPON_CODE = SAVE100
USER_SEGMENT = VIP
```

## 7.2 ทำไมต้องแยก Condition ออกจาก Strategy

เพราะสองเรื่องนี้คนละหน้าที่:

```
Condition = ใช้ได้ไหม  
Action    = ลดเท่าไร
```

ถ้าเอา Condition ไปไว้ใน Strategy จะทำให้ Strategy ใหญ่เกินไปและซ้ำซ้อน

ตัวอย่างที่ไม่ดี:

```
PercentageDiscountStrategy:  
    check product_id  
    check category  
    check coupon  
    calculate discount
```

ตัวอย่างที่ดี:

```
ConditionEvaluator:  
    check product_id  
    check category  
    check coupon  
  
PercentageDiscountStrategy:  
    calculate percentage discount
```

## 7.3 Specification Examples

| Specification         | หน้าที่             |
|-----------------------|---------------------|
| ProductConditionSpec  | ตรวจสอบ Product ID  |
| CategoryConditionSpec | ตรวจสอบ Category    |
| MinOrderAmountSpec    | ตรวจสอบยอดขั้นต่ำ   |
| CouponCodeSpec        | ตรวจสอบ Coupon Code |
| DateRangeSpec         | ตรวจสอบช่วงเวลา     |



|                 |                 |
|-----------------|-----------------|
| Specification   | หน้าที่         |
| UserSegmentSpec | ตรวจกลุ่มลูกค้า |

## 8. Chain / Pipeline Execution

### 8.1 ใช้ทำอะไร

ใช้ Apply Promotion ตามลำดับที่กำหนด

Pipeline:

```

Validate
→ Load Promotions
→ Filter Eligible
→ Sort Promotions
→ Apply Promotion 1
→ Apply Promotion 2
→ Apply Promotion 3
→ Generate Result

```

### 8.2 ทำไมต้องมี Pipeline

เพราะ Promotion ซ้อนกันอาจให้ผลลัพธ์ต่างกันตามลำดับ

ตัวอย่าง:

```

Product price = 1,000

Promo A = ลด 10%
Promo B = ลด 100 บาท

```

ถ้า A ก่อน B:

```

1,000 - 10% = 900
900 - 100 = 800

```

ถ้า B ก่อน A:

```
1,000 - 100 = 900
900 - 10% = 810
```

ผลลัพธ์ไม่เท่ากัน

ดังนั้นระบบต้องกำหนด Flow ที่ชัดเจน:

```
scope_order ASC
priority ASC
created_at ASC
promotion_id ASC
```

## 9. Policy Pattern: Stacking Policy

### 9.1 ใช้ทำอะไร

ใช้ควบคุมว่า Promotion หลายตัวใช้ร่วมกันได้หรือไม่

Field สำคัญ:

```
stackable
exclusive
stop_processing
priority
scope
conflict_group
```

### 9.2 Stacking Rules

| Rule                                | Behavior                                                |
|-------------------------------------|---------------------------------------------------------|
| <code>stackable = true</code>       | ใช้ร่วมกับ Promotion อื่นได้                            |
| <code>stackable = false</code>      | ถ้ามี Promotion ใน Conflict เดียวกันถูกใช้แล้ว ให้ Skip |
| <code>exclusive = true</code>       | ใช้แล้วหยุด Promotion ที่ขัดแย้ง                        |
| <code>stop_processing = true</code> | ใช้แล้วหยุด Rule ถัดไป                                  |

| Rule                       | Behavior                          |
|----------------------------|-----------------------------------|
| <code>scope stopped</code> | Promotion ใน Scope นั้นจะถูก Skip |

## 9.3 Example: Stackable

Product price = 1,000

Promo A:

ลด 10%

priority = 1

stackable = true

Promo B:

ลด 100 บาท

priority = 2

stackable = true

Calculation:

$1,000 - 10\% = 900$

$900 - 100 = 800$

## 9.4 Example: Exclusive

Product price = 1,000

Promo A:

ลด 10%

priority = 1

exclusive = true

Promo B:

ลด 100 บาท

priority = 2

Calculation:

```
Promo A applied  
Final = 900  
  
Promo B skipped  
reason = scope_stopped
```

## 10. Repository Pattern

### 10.1 ใช้ทำอะไร

ใช้แยก Database Access ออกจาก Business Logic

Repository หลัก:

```
ProductRepository  
PromotionRepository  
OrderRepository  
CalculationLogRepository  
PromotionUsageRepository
```

### 10.2 ทำไมต้องใช้ Repository

เพราะ Promotion Engine ไม่ควรรู้ว่า Data มาจาก:

```
MySQL  
Redis  
Mock  
External Service
```

Engine ควรรู้ผ่าน Interface เท่านั้น

ข้อดี:

- Test ง่าย
- Mock ได้
- เปลี่ยน Query ได้โดยไม่กระทบ Engine

- Business Logic ไม่ผูกกับ GORM
- 

## 11. Context Object Pattern

### 11.1 ใช้ทำอะไร

ใช้เก็บ State ทั้งหมดระหว่างการคำนวณ

ข้อมูลใน Calculation Context:

```
user_id
coupon_code
items
original_total
discount_total
final_total
applied_promotions
skipped_promotions
stopped_scopes
```

### 11.2 ทำไมต้องมี Context

เพราะระหว่างคำนวณต้องรู้ว่า:

- ราคาตั้งต้นคือเท่าไร
- ลดไปแล้วเท่าไร
- เหลือเท่าไร
- Promotion ไหน Apply แล้ว
- Promotion ไหนถูก Skip
- Scope ไหนถูก Stop
- Breakdown เป็นอย่างไร

ถ้าไม่มี Context Logic จะกระจายและ Debug ยาก

---

## 12. Algorithm Overview

Algorithm หลักของ Promotion Engine:

1. รับ Calculation Context
2. โหลด Active Promotions
3. Filter Promotion ที่ไม่เกี่ยวข้อง
4. ตรวจสอบ Conditions และ Targets
5. Group Promotion ตาม Scope
6. Sort Promotion ตาม Scope และ Priority
7. Apply Promotion ทีละตัว
8. ตรวจสอบ Stacking Policy
9. Cap Discount ไม่ให้เกินยอดคงเหลือ
10. บันทึก Applied / Skipped Breakdown
11. Validate Final Amount
12. Return Calculation Result

## 13. Promotion Sorting Algorithm

### Sort Rule

```
scope_order ASC  
priority ASC  
created_at ASC  
promotion_id ASC
```

Scope Order:

```
ITEM = 1  
CART = 2  
COUPON = 3  
SHIPPING = 4
```

Priority Rule:

```
priority น้อยกว่า = คำนวณก่อน
```

Tie-breaker:

ถ้า priority เท่ากัน ใช้ created\_at และ promotion\_id

เหตุผล:

ทำให้ผลลัพธ์เป็น Deterministic

## 14. Eligibility Algorithm

Promotion ใช้ได้เมื่อ:

```
status = ACTIVE
starts_at <= now
ends_at >= now
target ตรงกับ order หรือ item
condition ผ่าน
usage limit ยังไม่เต็ม
coupon code ถูกต้อง ถ้ามี
```

ถ้าไม่ผ่านต้องเพิ่มเข้า `skipped_promotions`

ตัวอย่าง Reason:

```
inactive
not_started
expired
target_not_matched
condition_not_matched
usage_limit_reached
coupon_not_matched
```

## 15. Main Calculation Algorithm

```
Calculate(context):
1. promotions = LoadActivePromotions(now)
```

```

2. eligible_promotions = []

3. for each promotion:
    if IsEligible(context, promotion):
        eligible_promotions.add(promotion)
    else:
        context.skip(promotion, reason)

4. SortPromotions(eligible_promotions)

5. for each promotion in eligible_promotions:

    if context.is_scope_stopped(promotion.scope):
        context.skip(promotion, "scope_stopped")
        continue

    if not stacking_policy.can_apply(context, promotion):
        context.skip(promotion, "stacking_conflict")
        continue

    strategy = strategy_factory.get(promotion.action_type)

    if strategy not found:
        context.skip(promotion, "strategy_not_found")
        continue

    result = strategy.apply(context, promotion)

    discount = min(result.discount, remaining_amount)

    context.apply_discount(result)

    context.add_breakdown(result)

    if promotion.exclusive or promotion.stop_processing:
        context.stop_scope(promotion.scope)

6. context.validate()

7. return context.result()

```

## 16. Discount Cap Algorithm

เพื่อป้องกันราคาติดลบ:



```
actual_discount = min(calculated_discount, remaining_amount)
```

ถ้า Discount ตีลบหรือ Config ผิด:

```
skip promotion  
reason = invalid_config
```

ตัวอย่าง:

```
Product price = 80  
Discount = 100  
  
actual_discount = min(100, 80)  
actual_discount = 80  
  
final = 0
```

## 17. Percentage Discount Algorithm

ใช้สำหรับลดแบบเปอร์เซ็นต์

Formula:

```
discount = amount * value_basis_points / 10000  
actual_discount = min(discount, amount)
```

ตัวอย่าง:

```
amount = 100000 satang  
value_basis_points = 1000  
  
discount = 100000 * 1000 / 10000  
discount = 10000 satang  
  
final = 90000 satang
```

แปลว่า:

1,000 บาท ลด 10% เหลือ 900 บาท

## 18. Fixed Amount Discount Algorithm

ใช้สำหรับลดแบบจำนวนเงิน

Formula:

```
discount = value_amount  
actual_discount = min(discount, remaining_amount)
```

ตัวอย่าง:

```
amount = 50000 satang  
value_amount = 10000 satang  
  
discount = 10000  
final = 40000
```

แปลว่า:

500 บาท ลด 100 บาท เหลือ 400 บาท

## 19. Item-level Promotion Algorithm

ใช้กับ Promotion ระดับสินค้า

Flow:

```
for each item:  
    find matched item promotions  
    sort by priority  
    apply each promotion
```

```
update item discount_amount
update item final_amount
```

ตัวอย่าง:

```
Product 1:
original = 100000
discount 10% = 10000
final = 90000
```

## 20. Cart-level Promotion Algorithm

ใช้กับยอดรวมหลัง Item-level Discount

Flow:

```
cart_subtotal = sum(item.final_amount)
discount = calculate cart discount
final_total = cart_subtotal - discount
```

ตัวอย่าง:

```
Product 1 final = 900
Product 2 final = 400

cart_subtotal = 1,300

Cart Discount = 100

final_total = 1,200
```

## 21. Algorithm Complexity

สมมติ:

```
n = จำนวน item ใน order
p = จำนวน active promotions
e = จำนวน eligible promotions
```

แบบ Optimize ด้วย Map:

```
Build promotion maps: O(p)
Lookup product promotion: O(1)
Lookup category promotion: O(1)
Sort eligible promotions: O(e log e)
Apply only matched promotions
```

เป้าหมาย:

```
ห้าม Query Database ใน Loop
ลดการ Scan Promotion ที่ไม่เกี่ยวข้อง
```

## 22. End-to-End Example ตามโจทย์

### Input

```
Product 1 ราคา 1,000 บาท
Product 2 ราคา 500 บาท
```

### Promotions

```
Promotion A:
Product 1 ลด 10%
scope = ITEM
priority = 1
stackable = true
action_type = PERCENTAGE_DISCOUNT

Promotion B:
Product 2 ลด 100 บาท
scope = ITEM
```

```
priority = 1
stackable = true
action_type = FIXED_AMOUNT_DISCOUNT
```

## Calculation

```
Product 1:
original = 1,000
discount = 1,000 * 10% = 100
final = 900

Product 2:
original = 500
discount = 100
final = 400
```

## Result

```
original_total = 1,500
discount_total = 200
final_total = 1,300
```

## 23. Error Handling Algorithm

Promotion Engine ต้องไม่ Panic เมื่อ Config ผิด

| Case                      | Behavior          |
|---------------------------|-------------------|
| Strategy not found        | Skip + Log        |
| Invalid percentage > 100% | Skip + Log        |
| Fixed amount <= 0         | Skip + Log        |
| Promotion ไม่มี Action    | Skip + Log        |
| Target ไม่ตรง             | Skip พร้อม Reason |
| Final price ติดลบ         | Cap ที่ 0         |

ตัวอย่าง Reason:

```
strategy_not_found
invalid_percentage
invalid_discount_amount
action_not_found
target_not_matched
scope_stopped
```

## 24. Test Cases ที่ Algorithm ต้องผ่าน

| Case                 | Expected Result   |
|----------------------|-------------------|
| Product 1 ลด 10%     | 1,000 → 900       |
| Product 2 ลด 100 บาท | 500 → 400         |
| Discount มากกว่าราคา | Final = 0         |
| ลด 10% แล้วลด 100    | 1,000 → 900 → 800 |
| ลด 100 แล้วลด 10%    | 1,000 → 900 → 810 |
| Exclusive Promotion  | ตัวถัดไปถูก Skip  |
| Stop Processing      | ตัวถัดไปถูก Skip  |
| Expired Promotion    | Skip              |
| Strategy Not Found   | Skip ไม่ Panic    |
| Priority เท่ากัน     | ใช้ Tie-breaker   |

## 25. Why This Design Works

**คำถาม: Promotion ซ้อนกันจัดลำดับยังไง?**

ตอบ:

```
ใช้ scope_order + priority + tie-breaker
แล้ว Apply ทีละตัวผ่าน Pipeline
ควบคุมด้วย stackable, exclusive, stop_processing
```

## คำถาม: เพิ่ม Promotion โดยไม่กระทบ Logic เดิมได้อย่างไร?

ตอบ:

```
เพิ่ม Strategy ใหม่
Register เข้า Factory
เพิ่ม Config ใน Database
เพิ่ม Test
ไม่ต้องแก้ Flow หลักของ Promotion Engine
```

## คำถาม: Promotion ใหม่ที่ไม่เคยมีมาก่อนรองรับยังไง?

ตอบ:

```
Database รองรับด้วย action_type และ value_json
Code รองรับด้วย Strategy ใหม่
Condition รองรับด้วย Specification ใหม่
```

ตัวอย่าง:

```
BUY_X_GET_Y
- เพิ่ม action_type
- เก็บ config ใน value_json
- เพิ่ม BuyXGetYStrategy
- เพิ่ม Unit Test
```

## 26. Final Design Decision

ระบบนี้ควรใช้:

```
Rule-based Promotion Engine
+ Strategy Pattern
+ Factory / Registry Pattern
+ Specification Pattern
+ Pipeline Execution
+ Stacking Policy
```

- + Repository Pattern
- + Calculation Context

Algorithm หลัก:

```
Validate Context
→ Load Active Promotions
→ Evaluate Eligibility
→ Sort by Scope and Priority
→ Apply Promotion Strategy
→ Apply Stacking Policy
→ Cap Discount
→ Generate Breakdown
→ Return Final Result
```

## สรุปสั้นสำหรับใส่สไลด์

Promotion Engine ใช้ Strategy Pattern เพื่อแยก Logic ของ Promotion แต่ละประเภท ใช้ Factory ในการเลือก Strategy จาก action\_type ใช้ Specification ตรวจสอบ Condition และใช้ Pipeline ที่เรียงตาม Scope และ Priority เพื่อให้การคำนวณ Promotion ชัดเจน เป็น Deterministic ระบบสามารถเพิ่ม Promotion ใหม่ได้โดยเพิ่ม Strategy และ Config ใหม่โดยไม่ต้องแก้ Calculation Flow หลัก.